



RAPPORT DE STAGE

DÉVELOPPEMENT D'UNE PLATEFORME E-COMMERCE SUR-MESURE

Présenté par :
Taha Hassbi

Entreprise d'accueil :
BizOps Consulting — Casablanca

Tuteur en entreprise : M. ZITOUNI Hamza

Encadrant académique : M. ABOUALI

Dédicaces

Je dédie ce travail...

À mes parents, pour leur soutien inconditionnel, leurs sacrifices et leur amour qui m'ont toujours poussé à donner le meilleur de moi-même.

À mon frère Soufiane, pour ses encouragements constants.

À mes amis et collègues de la filière Développement des Applications Informatiques (DAI), avec qui j'ai partagé des moments d'apprentissage inoubliables.

Taha Hassbi

Remerciements

Avant d'entamer la présentation de ce rapport, je tiens à exprimer ma profonde gratitude envers toutes les personnes qui ont contribué, de près ou de loin, au bon déroulement de mon stage et à la réalisation de ce projet.

Mes remerciements s'adressent en premier lieu à mon encadrant académique, **M. ABOUALI**, pour son suivi, ses précieux conseils et sa disponibilité tout au long de ma formation au Centre BTS Al Kendi.

Je tiens également à remercier chaleureusement la direction et toute l'équipe de **BizOps Consulting** pour leur accueil, leur confiance et l'environnement professionnel stimulant qu'ils m'ont offert. Cette expérience m'a permis de mettre en pratique mes connaissances théoriques dans des conditions réelles et exigeantes.

Enfin, je remercie le corps professoral de la filière DAI pour la qualité de l'enseignement dispensé au cours de ces deux années.

Table des matières

Dédicaces

Remerciements 1

Introduction Générale 5

1 Contexte et Environnement de Travail 7

1.1 Présentation de l'Organisme d'Accueil 7

1.2 Présentation du Projet "Deer Goods" 7

1.2.1 Problématique 7

1.2.2 Objectifs du Projet 8

1.3 Environnement de Développement et Outils 8

2 Analyse et Spécifications des Besoins 9

2.1 Spécifications Fonctionnelles 9

2.1.1 Espace Client 9

2.1.2 Espace Administrateur 9

2.2 Spécifications Non-Fonctionnelles 10

2.3 Modélisation par les Cas d'Utilisation (UML) 10

3 Conception et Architecture 12

3.1 Architecture de l'Application 12

3.2 Modélisation de la Base de Données 12

3.3 Conception Dynamique : Le Processus "Custom Orders" 13

3.4 Spécifications des Points d'Accès de l'API (REST Endpoints) .. 14

3.4.1 Gestion de l'Authentification 14

3.4.2 Gestion du Catalogue et Produits 15

3.4.3 Gestion des Commandes Sur-Mesure 15

4 Réalisation et Implémentation 16

4.1 L'Interface Client 16

4.1.1 Page d'Accueil et Boutique 16

4.1.2 Le Tunnel de Commande Sur-Mesure 17

4.2	Espace Administration et Logique Métier	18
4.2.1	Le Dashboard Administrateur	18
4.2.2	Gestion Automatisée des Stocks et Commandes	19
5	Déploiement, Sécurité et Bilan	20
5.1	Politiques de Sécurité et Authentification	20
5.1.1	Row Level Security (RLS) avec Supabase	20
5.1.2	Authentification et Sessions (JWT)	20
5.2	Intégrations Métier : Facturation et Communication	21
5.2.1	Génération Automatisée de Factures (PDF)	21
5.2.2	Intégration de l'API WhatsApp	21
5.3	Bilan du Stage Professionnel	21
5.3.1	Bilan Technique	21
5.3.2	Bilan Personnel et Professionnel	22
	Webographie et Outils	23
	Conclusion Générale	25
	Annexes	25
A	Extraits de Code et Logique Métier	26
B	Interfaces et Fonctionnalités Supplémentaires	29

Table des figures

2.1	Diagramme des cas d'utilisation du système Deer Goods	11
3.1	Diagramme de classes (Entité-Relation) de la base de données	13
3.2	Diagramme de séquence du processus des commandes sur-mesure	14
4.1	Interface de la page Boutique avec filtres dynamiques	17
4.2	Interface de soumission d'une commande sur-mesure	17
4.3	Tableau de bord de l'Administrateur	18
B.1	Interface du Panier (Cart) et récapitulatif de commande	29
B.2	Modèle de Facture PDF générée par le système	30
B.3	Aperçu de la page d'accueil sur un terminal mobile (Iphone XR)	31
B.4	Interface de création de compte (Sign Up / Register)	32

Introduction Générale

À l'ère de la transformation numérique, le commerce électronique (e-commerce) n'est plus une simple alternative, mais une nécessité absolue pour toute entreprise souhaitant pérenniser son activité. Cependant, face à la multiplication des plateformes standards et uniformisées, les marques artisanales et premium cherchent à se démarquer en offrant une expérience utilisateur unique, reflétant la qualité de leurs produits.

C'est dans ce contexte que s'inscrit mon projet de stage, réalisé au sein du cabinet d'expertise technologique **BizOps Consulting**. L'objectif principal de ce stage a été la conception et le développement *from scratch* d'une plateforme e-commerce sur-mesure pour la marque "**Deer Goods**". Spécialisée dans la maroquinerie haut de gamme et les articles en cuir cousus à la main, la marque nécessitait une solution digitale capable de gérer non seulement des ventes classiques, mais aussi un flux complexe de commandes personnalisées (Bespoke/Custom Orders).

Le présent rapport détaille les différentes phases de ce projet, allant de l'analyse des besoins à la mise en production. Il est structuré autour de cinq chapitres principaux :

- **Le premier chapitre** est consacré à la présentation du contexte général du projet, de l'organisme d'accueil BizOps Consulting, ainsi que de l'environnement de travail.
- **Le deuxième chapitre** aborde l'analyse et la spécification des besoins, modélisées à travers le langage UML pour définir clairement les contours fonctionnels de l'application.
- **Le troisième chapitre** détaille la conception technique, les choix architecturaux (Stack PERN) et la structuration de la base de données avec l'inventaire complet des points d'accès de l'API.
- **Le quatrième chapitre** présente la phase de réalisation, en mettant en évidence les solutions implémentées pour la gestion dynamique des stocks, l'interface utilisateur, et le tunnel de commande sur-mesure.

- **Le cinquième chapitre** conclut avec la phase de test, les politiques de sécurité, ainsi que le bilan personnel et professionnel de ce stage.

Ce document retrace ainsi mon immersion dans le monde du développement web moderne, où les défis techniques ont côtoyé les exigences de l'expérience utilisateur et des règles métiers du commerce premium.

1. Contexte et Environnement de Travail

1.1. Présentation de l'Organisme d'Accueil

BizOps Consulting est un cabinet d'expertise technologique basé à Casablanca. La société se positionne comme un partenaire stratégique pour les entreprises souhaitant accélérer leur transformation digitale. En combinant conseil IT, optimisation des processus opérationnels et développement logiciel sur mesure, BizOps offre des solutions qui répondent directement aux problématiques complexes de ses clients.

Durant mon stage, j'ai été intégré au sein du pôle de développement web, où l'accent est mis sur l'utilisation de technologies modernes et de méthodologies agiles pour livrer des produits robustes et évolutifs.

1.2. Présentation du Projet "Deer Goods"

Le projet qui m'a été confié concerne la digitalisation de la marque **Deer Goods**. Il s'agit d'un atelier spécialisé dans la fabrication d'articles en cuir véritable (Full-grain Leather), faits à la main.

1.2.1. Problématique

L'atelier fonctionnait principalement via des canaux traditionnels et des réseaux sociaux (Instagram). La gestion des commandes, notamment les commandes sur-mesure (Custom Orders) où le client demande un design spécifique, devenait fastidieuse. Les échanges de devis, la validation des croquis et le suivi des paiements entraînaient des pertes de temps et des erreurs de gestion de stock.

1.2.2. Objectifs du Projet

L'objectif était de développer une application web complète permettant de :

- Exposer le catalogue de produits prêts-à-porter avec des filtres dynamiques.
- Digitaliser intégralement le processus de **demande sur-mesure** (Upload d'images, devis, acceptation, et création automatique de produit).
- Automatiser la gestion des stocks en temps réel.
- Faciliter le traitement administratif via la génération de factures PDF et le contact direct par WhatsApp.

1.3. Environnement de Développement et Outils

Pour garantir l'efficacité de la phase de réalisation, j'ai mis en place un environnement de travail optimisé.

- **Matériel et IDE** : L'ensemble du développement a été réalisé sur une machine équipée d'un processeur Intel Core i5-13420H, assurant la fluidité nécessaire pour faire tourner simultanément le serveur backend, le frontend, et la base de données. J'ai utilisé **Visual Studio Code** comme éditeur de texte principal.
- **Gestion de version** : L'outil **Git** et la plateforme **GitHub** ont été utilisés pour le versioning du code, permettant un suivi précis des évolutions (*commits*) et une gestion sécurisée des modifications.

2. Analyse et Spécifications des Besoins

La phase d'analyse est une étape cruciale dans le cycle de vie du développement logiciel. Elle permet de définir ce que le système doit faire avant de décider comment il va le faire.

2.1. Spécifications Fonctionnelles

Les besoins fonctionnels décrivent les actions que le système doit être capable d'exécuter. Nous avons divisé ces besoins en deux catégories : ceux destinés aux clients et ceux destinés à l'administration.

2.1.1. Espace Client

- **Authentification** : Inscription, connexion et gestion du profil utilisateur de manière sécurisée.
- **Catalogue** : Consultation des articles avec filtrage par prix et par catégorie.
- **Panier d'achat** : Ajout, modification de quantité et suppression d'articles en vérifiant la disponibilité en stock.
- **Commandes Sur-Mesure (Custom Orders)** : Possibilité de soumettre une description détaillée et une image pour un article personnalisé, et de suivre le statut du devis.

2.1.2. Espace Administrateur

- **Gestion des Produits** : Opérations CRUD (Création, Lecture, Mise à jour, Suppression) sur les produits.

- **Gestion du Catalogue** : Opérations CRUD sur les catégories.
- **Gestion des Stocks** : Suivi précis des quantités.
- **Traitement des Commandes** : Validation, annulation (avec restitution automatique du stock), génération de factures PDF.
- **Gestion des Devis** : Analyse des demandes sur-mesure et proposition de prix (Quote).

2.2. Spécifications Non-Fonctionnelles

- **Performance** : Le système doit être réactif, avec des temps de chargement minimes grâce à une architecture en Single Page Application (SPA).
- **Sécurité** : Protection des routes via des middlewares (Vérification des rôles), mots de passe hachés, et sécurisation des requêtes à la base de données.
- **Ergonomie (UI/UX)** : L'interface doit refléter l'image "Premium" de la marque (Design épuré, animations fluides, responsive design pour mobiles et tablettes).

2.3. Modélisation par les Cas d'Utilisation (UML)

Pour formaliser les besoins, nous avons utilisé le diagramme de cas d'utilisation. Il met en évidence les interactions entre les acteurs (Client, Administrateur) et le système.

Il est important de noter la dépendance stricte (relation «**include**») vers le cas d'utilisation "S'authentifier". Cette conception garantit que des actions critiques telles que l'ajout au panier ou la demande de devis ne peuvent être effectuées que par un utilisateur identifié.

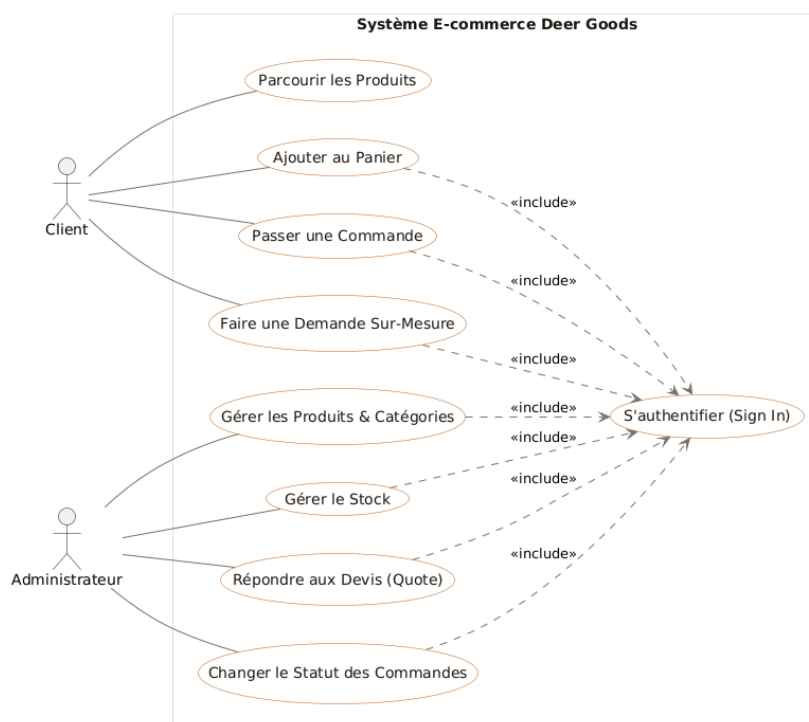


FIGURE 2.1 – Diagramme des cas d'utilisation du système Deer Goods

3. Conception et Architecture

La phase de conception permet de définir l'architecture technique globale et la structure des données avant de passer au développement physique de l'application. Cette étape garantit la scalabilité et la robustesse du système.

3.1. Architecture de l'Application

Pour répondre aux exigences de performance et de sécurité, nous avons opté pour une architecture basée sur la pile **PERN** (PostgreSQL, Express, React, Node.js), enrichie par **Supabase** (BaaS) pour la gestion de la base de données et du stockage.

L'architecture est divisée en trois couches principales :

- **Couche Présentation (Frontend)** : Développée en **React.js** avec **TypeScript**. Elle offre une interface utilisateur réactive (Single Page Application) et utilise **Framer Motion** pour des animations fluides reflétant l'aspect "Premium" de la marque.
- **Couche Métier (Backend REST API)** : Développée avec **Node.js** et **Express.js**. Elle centralise la logique métier complexe : validation des requêtes, génération de factures PDF, et sécurisation des routes via des middlewares (Vérification de Token JWT).
- **Couche Accès aux Données** : Gérée par **PostgreSQL** via **Supabase**, qui assure la persistance des données relationnelles et le stockage des fichiers médias.

3.2. Modélisation de la Base de Données

Le diagramme de classes (Modèle Logique de Données) illustre la structure de notre base de données. Il met en évidence les relations clés, notamment entre les *Utilisateurs*, les *Commandes*, les *Produits* et les *Catégories*.

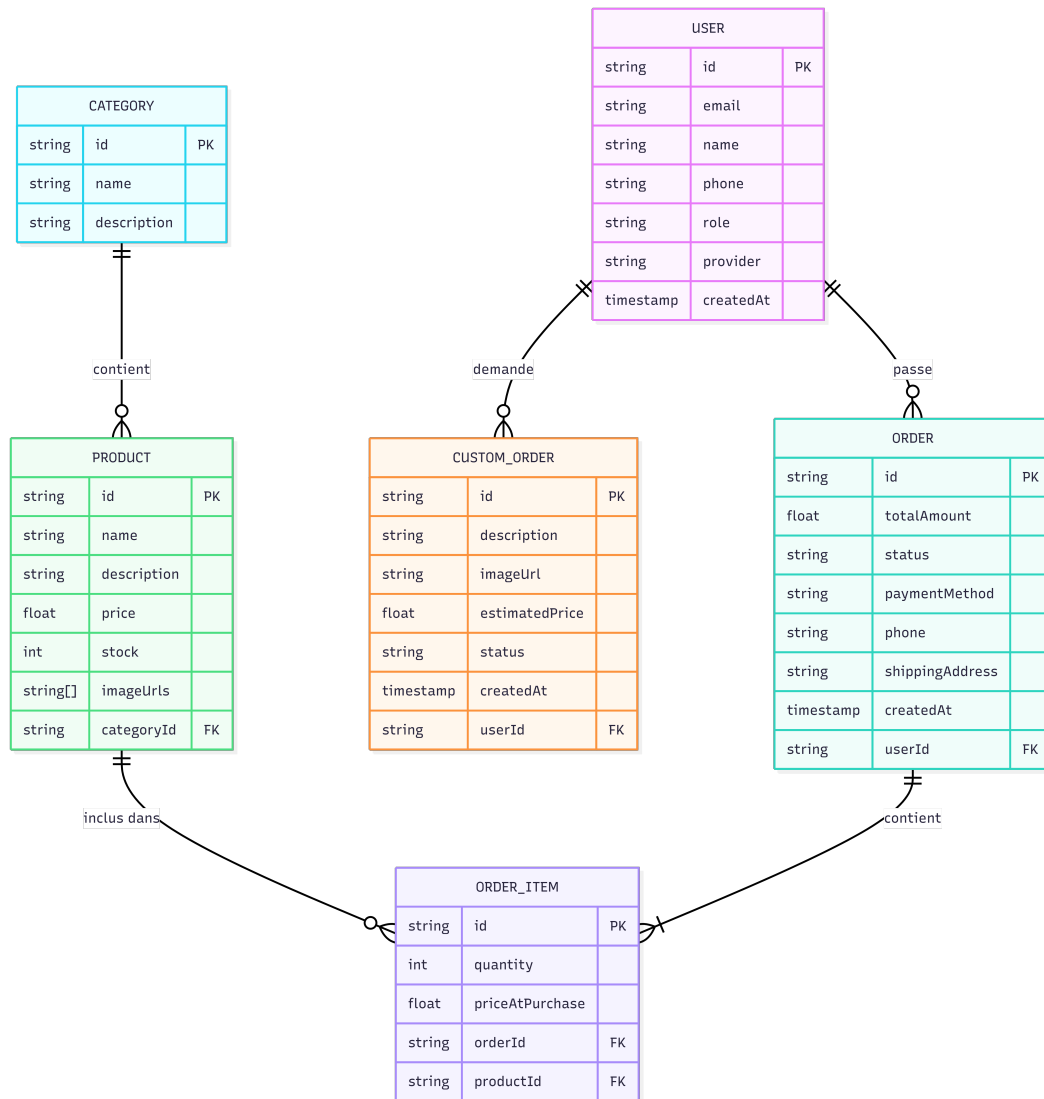


FIGURE 3.1 – Diagramme de classes (Entité-Relation) de la base de données

3.3. Conception Dynamique : Le Processus "Custom Orders"

L'une des fonctionnalités phares de l'application est la gestion des commandes sur-mesure (Custom Orders). Afin de clarifier les interactions entre le client, l'interface, le serveur API et la base de données lors de ce processus, nous avons modélisé le diagramme de séquence suivant.

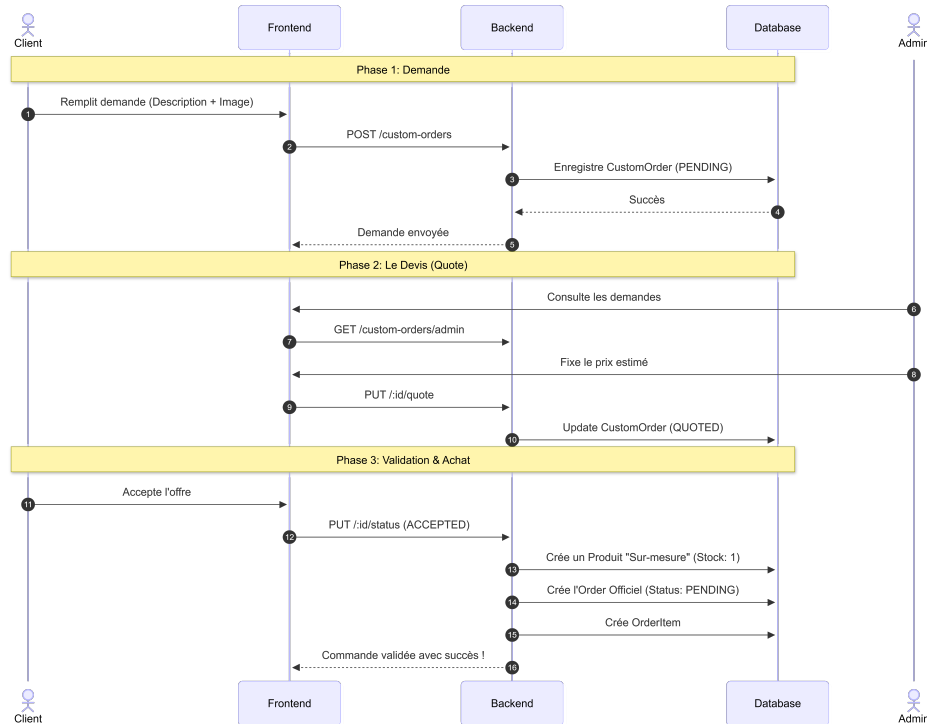


FIGURE 3.2 – Diagramme de séquence du processus des commandes sur-mesure

3.4. Spécifications des Points d'Accès de l'API (REST Endpoints)

Afin de documenter la communication entre la couche de présentation et la couche métier, nous récapitulons ci-dessous l'ensemble des routes HTTP développées au sein de notre serveur Express.

3.4.1. Gestion de l'Authentification

Méthode	Route	Description	Accès
POST	/api/auth/register	Inscription d'un nouvel utilisateur	Public
POST	/api/auth/login	Connexion et génération du token JWT	Public

TABLE 3.1 – Endpoints de gestion de l'authentification

3.4.2. Gestion du Catalogue et Produits

Méthode	Route	Description	Accès
GET	/api/products	Récupération de tous les articles filtrés	Public
GET	/api/categories	Liste des catégories (hors Custom)	Public
POST	/api/products	Création manuelle d'une pièce	Admin
PUT	/api/products/:id	Modification du stock ou du prix	Admin

TABLE 3.2 – Endpoints du catalogue de produits

3.4.3. Gestion des Commandes Sur-Mesure

Méthode	Route	Description	Accès
POST	/api/custom-orders	Soumission d'une description et image	Client
GET	/api/custom-orders/my-requests	Consultation de l'historique personnel	Client
GET	/api/custom-orders/admin	Visualisation de toutes les demandes	Admin
PUT	/api/custom-orders/:id/quote	Envoi du prix estimé (Devis) au client	Admin
PUT	/api/custom-orders/:id/status	Validation finale et création de commande	Client/Admin

TABLE 3.3 – Endpoints liés au processus sur-mesure

4. Réalisation et Implémentation

Ce chapitre présente l'aboutissement technique du projet. Nous y exposons les interfaces graphiques développées, conçues pour optimiser l'expérience utilisateur, ainsi que la logique métier implémentée côté serveur.

4.1. L'Interface Client

L'objectif principal du frontend était de retranscrire l'identité visuelle de l'atelier de maroquinerie, tout en offrant une navigation intuitive.

4.1.1. Page d'Accueil et Boutique

La page boutique (Shop) permet aux utilisateurs de parcourir les articles grâce à des filtres dynamiques (par catégorie et par fourchette de prix). Une vérification en temps réel du stock est effectuée : si un produit est épuisé, un badge "Sold Out" s'affiche et le bouton d'ajout au panier est désactivé.

L'interface de la page **Boutique** (illustrée par la Figure 4.1) utilise un algorithme de filtrage mémorisé via le hook React `useMemo`. Cela permet de recalculer instantanément l'affichage dès que l'utilisateur modifie la catégorie de maroquinerie ou la tranche de prix sélectionnée, évitant ainsi des requêtes réseau redondantes vers la base de données. De plus, afin de protéger l'exclusivité des articles réguliers, une règle logique stricte est appliquée dans l'état local du composant : toute catégorie nommée "*Custom Orders*" ou tout produit contenant le mot-clé "*Commande*" est automatiquement masqué du catalogue public. Sur le plan ergonomique, le composant applique une grille fluide à deux colonnes sur les terminaux mobiles, assurant un affichage compact et harmonieux des fiches produits.

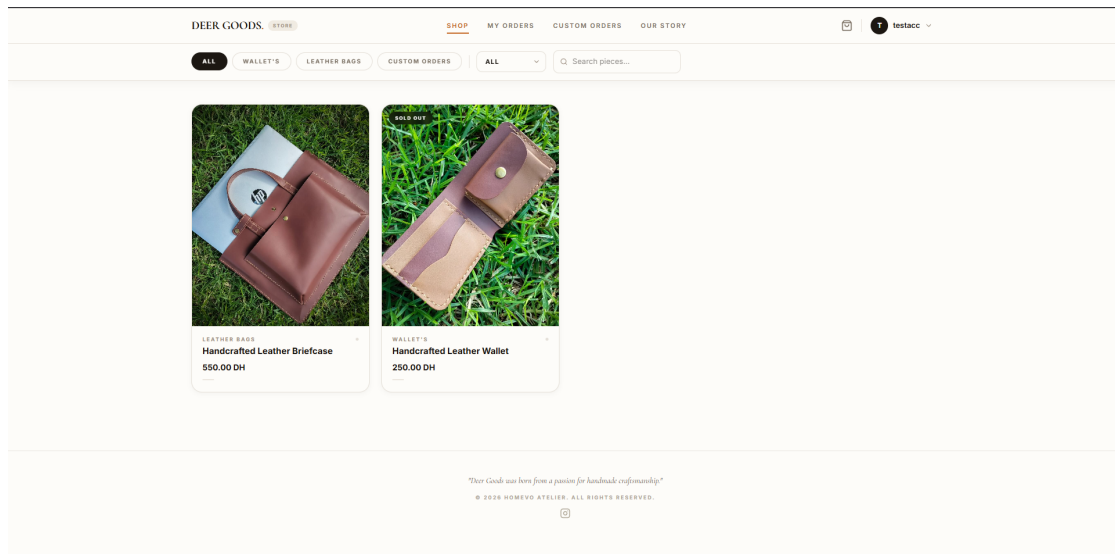


FIGURE 4.1 – Interface de la page Boutique avec filtres dynamiques

4.1.2. Le Tunnel de Commande Sur-Mesure

Le client dispose d'une interface dédiée pour soumettre une demande spécifique ne figurant pas dans le catalogue standard. Il peut y uploader une image d'inspiration et décrire son besoin. Cette requête est envoyée au backend via un objet `FormData` pour gérer le fichier image.

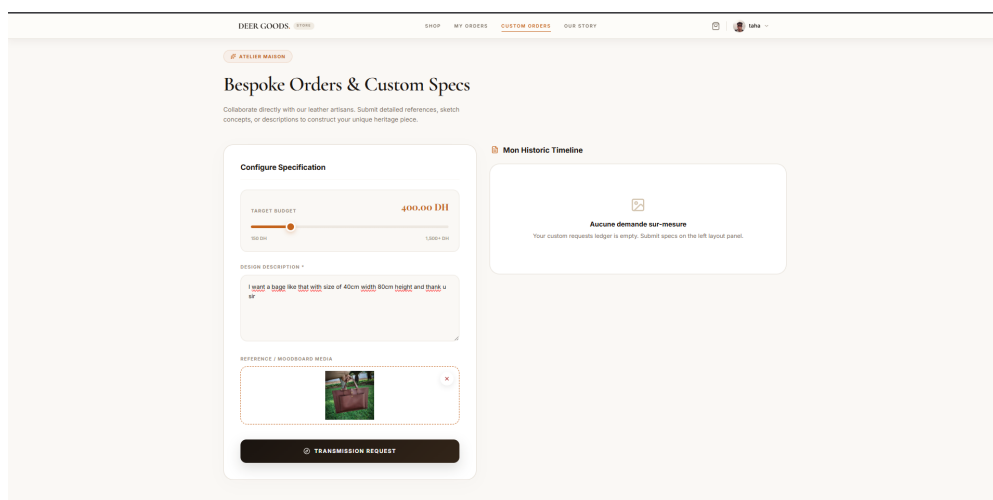


FIGURE 4.2 – Interface de soumission d'une commande sur-mesure

L'interface de soumission sur-mesure, présentée dans la Figure 4.2, est le point d'entrée du processus de personnalisation de l'atelier. Conçue comme un formulaire asynchrone sécurisé, elle oblige l'utilisateur à être préalablement connecté. Le champ de téléversement d'image accepte les fichiers graphiques (croquis, dimensions ou inspira-

tions) et génère une prévisualisation instantanée côté client. Lors de la soumission, les données textuelles et le flux binaire du fichier sont encapsulés dans un objet JavaScript `FormData`, puis envoyés vers la route HTTP dédiée. Le statut de la demande passe alors immédiatement à **PENDING**, permettant au client de suivre l'avancement de son dossier en temps réel depuis son espace personnel.

4.2. Espace Administration et Logique Métier

Le backend centralise l'automatisation des processus afin d'alléger la charge de travail des administrateurs de BizOps Consulting.

4.2.1. Le Dashboard Administrateur

Accessible uniquement aux utilisateurs disposant du rôle **ADMIN**, le tableau de bord permet de visualiser toutes les transactions. L'administrateur peut y fixer un devis (Quote) pour les commandes sur-mesure ou changer le statut des commandes standards (Pending, Shipped, Delivered, Canceled).

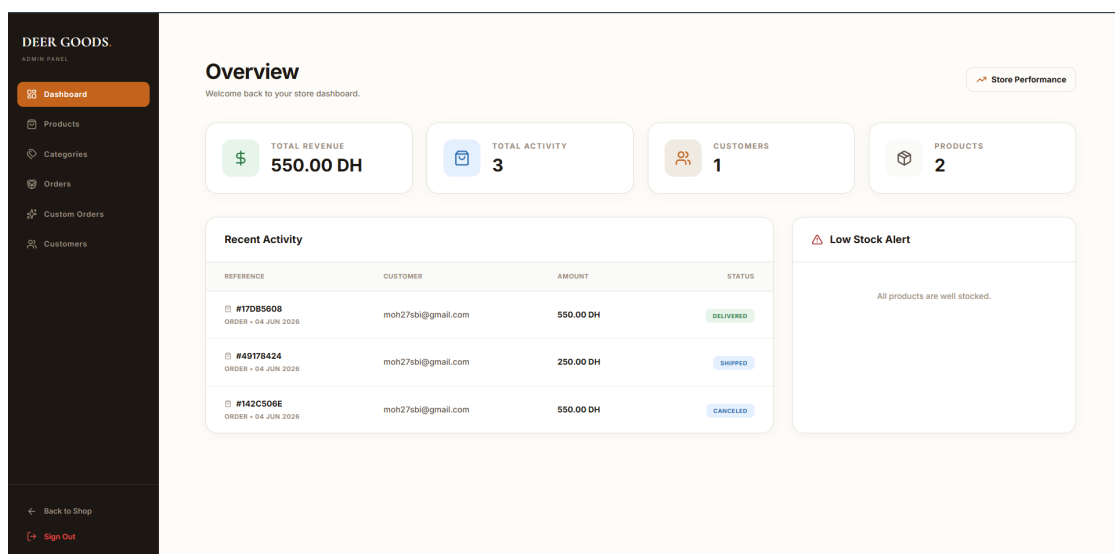


FIGURE 4.3 – Tableau de bord de l'Administrateur

Le **Dashboard Administrateur** (Figure 4.3) fait office de cockpit de contrôle pour la marque. Développé avec des restrictions d'accès strictes via notre middleware `isAdmin`, il affiche de manière centralisée les ventes courantes, l'état global du stock et la liste des devis en attente de prix. Pour chaque demande sur-mesure validée, l'administrateur saisit le montant estimé et déclenche l'envoi du devis en un seul clic,

faisant basculer l'état système du dossier vers la valeur `QUOTED`. Les interactions administratives intègrent également des boutons d'action rapide utilisant des masques d'URL dynamiques pour ouvrir instantanément l'interface native de WhatsApp Web avec un message pré-formaté.

4.2.2. Gestion Automatisée des Stocks et Commandes

Lorsqu'une commande sur-mesure est acceptée par le client après le devis, le système backend exécute une transaction logique complexe :

1. Récupération des informations du client.
2. Création automatique d'un produit "fantôme" (Bespoke Product) affecté à une catégorie cachée, avec un stock initialisé à 1.
3. Création immédiate de la commande officielle (`Order`) et de sa ligne de commande (`OrderItem`) liant le client à son produit sur-mesure.

Cette automatisation garantit que les commandes personnalisées entrent dans le circuit financier classique du e-commerce sans nécessiter de saisie manuelle redondante par l'administrateur.

5. Déploiement, Sécurité et Bilan

La dernière phase du projet a consisté à sécuriser la plateforme, à intégrer des fonctionnalités métier avancées pour l'administration, et à préparer l'application pour un environnement de production.

5.1. Politiques de Sécurité et Authentification

Dans une application e-commerce, la sécurité des données est primordiale.

5.1.1. Row Level Security (RLS) avec Supabase

Pour protéger la base de données, nous avons implémenté des politiques RLS (Row Level Security) directement sur PostgreSQL via Supabase. Cette approche garantit que les données ne sont accessibles qu'aux utilisateurs autorisés. Par exemple :

- **Table des Commandes (Order)** : Un client ne peut lire (SELECT) ou créer (INSERT) que ses propres commandes, identifiées par son `user_id`.
- **Rôle Administrateur** : Seuls les utilisateurs ayant le rôle `ADMIN` peuvent modifier les statuts des commandes ou mettre à jour le catalogue des produits.

5.1.2. Authentification et Sessions (JWT)

La gestion des sessions utilisateurs est basée sur des JSON Web Tokens (JWT). À chaque connexion réussie, un token sécurisé est généré, stocké localement, et transmis dans les en-têtes (Headers) de chaque requête HTTP vers notre API backend.

5.2. Intégrations Métier : Facturation et Communication

Pour faciliter le travail quotidien de l'atelier Deer Goods, deux intégrations majeures ont été développées.

5.2.1. Génération Automatisée de Factures (PDF)

Côté backend, lorsqu'une commande est validée, le système génère un document PDF contenant tous les détails de la transaction (Produits, prix, informations du client, et numéro de suivi). Ce fichier est ensuite stocké de manière sécurisée et un lien de téléchargement est mis à disposition de l'administrateur et du client.

5.2.2. Intégration de l'API WhatsApp

Étant donné que la clientèle de Deer Goods interagit fortement via les réseaux sociaux, nous avons mis en place des boutons de redirection directe vers WhatsApp. Ces liens pré-remplissent un message contenant le numéro de la commande pour un service client rapide et personnalisé.

5.3. Bilan du Stage Professionnel

Ce stage au sein de **BizOps Consulting** a été une véritable immersion dans le monde du développement logiciel professionnel.

5.3.1. Bilan Technique

D'un point de vue technique, ce projet m'a permis de consolider mes acquis académiques et d'explorer des technologies modernes. La conception d'une API RESTful robuste avec Node.js, couplée à une interface dynamique en React, m'a appris à gérer la complexité d'une application Full-Stack. L'utilisation de Supabase m'a également initié aux solutions Backend-as-a-Service (BaaS) modernes.

5.3.2. Bilan Personnel et Professionnel

Sur le plan professionnel, travailler sur un cas réel (la marque Deer Goods) m'a confronté aux exigences de qualité, de respect des délais et d'ergonomie. J'ai appris à traduire les besoins métier en spécifications techniques et à faire preuve d'autonomie face aux erreurs et aux bugs. L'accompagnement de mon tuteur en entreprise a été essentiel pour adopter les bonnes pratiques de codage et de gestion de projet.

Webographie et Outils

Dans le cadre de la réalisation de ce projet, plusieurs outils, frameworks et documentations officielles ont été consultés et utilisés. Cette section regroupe les principales ressources qui ont constitué l'environnement technique de l'application "Deer Goods".

Outils de Développement

- **Visual Studio Code** : Éditeur de code principal utilisé pour l'ensemble du développement (Frontend et Backend). (<https://code.visualstudio.com/>)
- **Git & GitHub** : Outils de gestion de version pour le suivi des modifications, le travail itératif et l'hébergement du code source.
- **Postman** : Plateforme de Test d'API, utilisée pour tester et valider les différents points de terminaison (Endpoints) du backend.
- **PlantUML & Mermaid.js** : Outils de modélisation pour la création et la génération des diagrammes UML (Cas d'utilisation, Classes, Séquence) par le code.

Technologies et Documentations Officielles

- **React.js** : Documentation officielle pour la création d'interfaces utilisateur réactives basées sur les composants. (<https://react.dev/>)
- **Supabase** : Alternative Backend-as-a-Service (BaaS) utilisée pour la base de données PostgreSQL, le stockage des images et l'authentification. (<https://supabase.com/docs>)

- **Framer Motion** : Bibliothèque d'animation pour React, essentielle pour l'intégration des transitions fluides et de l'aspect "Premium" de l'UI. (<https://www.framer.com/motion/>)
- **Node.js & Express.js** : Documentation technique pour la mise en place du serveur backend, le traitement des requêtes HTTP et le routage. (<https://expressjs.com/>)

Code Source du Projet

L'intégralité du code source de l'application e-commerce "Deer Goods" (incluant l'interface client, le dashboard administrateur et l'API backend), ainsi que l'historique complet des *commits*, sont disponibles et consultables publiquement sur mon dépôt GitHub.

Ce dépôt reflète l'évolution du projet, l'application des bonnes pratiques de développement, ainsi que la configuration des différents scripts de déploiement.

- **Dépôt GitHub (Code Source)** : <https://github.com/tahv7x/DEER-GOODS>

Conclusion Générale

La réalisation de la plateforme e-commerce "Deer Goods" durant mon stage chez BizOps Consulting représente l'aboutissement de ma formation en Développement des Applications Informatiques (DAI).

L'objectif principal était de digitaliser les processus de vente et de gestion d'un atelier de maroquinerie premium, en mettant un accent particulier sur la gestion des commandes sur-mesure (Custom Orders). En passant par toutes les phases du cycle de développement logiciel — de l'analyse des besoins et la modélisation UML, jusqu'à l'implémentation de l'interface utilisateur en React et du backend en Node.js — j'ai pu livrer une solution fonctionnelle, sécurisée et alignée avec l'image de la marque.

Ce projet m'a prouvé que le rôle d'un développeur ne se limite pas à écrire du code, mais consiste avant tout à concevoir des solutions logiques qui facilitent le travail humain et améliorent l'expérience client. L'automatisation de la gestion des stocks, la génération des factures, et le tunnel des devis sur-mesure ont démontré l'impact direct de la technologie sur l'efficacité opérationnelle d'une entreprise.

Enfin, cette expérience m'a donné confiance en mes capacités à mener à bien un projet complexe de bout en bout. Je suis désormais prêt à affronter de nouveaux défis professionnels et à poursuivre mon évolution dans le domaine de l'ingénierie logicielle.

A. Extraits de Code et Logique Métier

Annexe A.1 : Contrôleur des Commandes Sur-Mesure (Node.js)

Ce script illustre le cœur algorithmique du système de traitement des commandes sur-mesure (Custom Orders). Il expose les différents points d'accès de l'API permettant au client de soumettre son besoin (avec upload sécurisé d'image vers Supabase) et à l'administrateur de fixer un devis.

L'algorithme le plus notable réside dans la route `PUT /:id/status` : lorsqu'un devis est accepté par le client (`status === 'ACCEPTED'`), le serveur effectue une transaction automatisée. Il vérifie l'existence d'une catégorie masquée, instancie un produit sur-mesure exclusif au client (avec un stock de 1), et génère immédiatement la commande officielle (`Order`) et sa ligne de commande (`OrderItem`) associée.

```
import express, { type Request, type Response } from 'express';
import supabase from '../config/db.js';
import { authenticate, isAdmin } from '../middlewares/authMiddleware.js';
import { upload } from '../middlewares/uploadMiddleware.js';
import crypto from 'crypto';

const router = express.Router();

// Route : Validation et Automatisation de la commande sur-mesure
router.put('/:id/status', authenticate, async (req: Request, res: Response) => {
  try {
    const { id } = req.params;
    const { status } = req.body;

    const { data: customOrder, error: fetchError } = await supabase
      .from('CustomOrder').select('*').eq('id', id).single();

    if (fetchError || !customOrder) throw new Error("Custom order introuvable");

    // --- LOGIQUE METIER AVANCEE : Creation du produit fantome ---
```

```

if (status === 'ACCEPTED' && customOrder.status !== 'ACCEPTED') {

  const { data: userData } = await supabase
    .from('User').select('name, phone').eq('id', customOrder.userId).
    single();

  const clientName = userData?.name || 'Client';
  const dynamicProductName = `${clientName} Commande`;

  // 1. Verification/Creation de la categorie cachee
  let { data: customCategory } = await supabase
    .from('Category').select('id').eq('name', 'Custom Orders').single();

  if (!customCategory) {
    const { data: newCat, error: catErr } = await supabase
      .from('Category')
      .insert([
        {
          id: crypto.randomUUID(),
          name: 'Custom Orders',
          description: 'Cat gorie cach e '
        }
      ]).select().single();
    if (catErr) throw catErr;
    customCategory = newCat;
  }

  // 2. Creation du produit sur-mesure
  const productImages = customOrder.imageUrl ? [customOrder.imageUrl] : [];
  const { data: bespokeProduct, error: prodErr } = await supabase
    .from('Product')
    .insert([
      {
        id: crypto.randomUUID(),
        name: dynamicProductName,
        description: 'Commande sur-mesure valid e par ${clientName}.',
        price: 0,
        stock: 1,
        categoryId: customCategory.id,
        imageUrls: productImages
      }
    ]).select().single();

  if (prodErr) throw prodErr;

  // 3. Generation de la commande officielle
  const orderId = crypto.randomUUID();
  const { error: orderError } = await supabase.from('Order').insert([
    {
      id: orderId,
      userId: customOrder.userId,
      totalAmount: customOrder.estimatedPrice,
      paymentMethod: 'COD',
      status: 'PENDING',
      phone: userData?.phone || null,
      shippingAddress: '    confirmer via WhatsApp'
    }
  ]);

  if (orderError) throw orderError;

  // 4. Liaison via OrderItem
  const { error: itemError } = await supabase.from('OrderItem').insert([

```

```
        id: crypto.randomUUID(),
        orderId: orderId,
        productId: bespokeProduct.id,
        quantity: 1,
        priceAtPurchase: customOrder.estimatedPrice
      }]);

      if (itemError) throw itemError;
    }

    // Mise a jour finale du statut
    const { data: updatedData, error } = await supabase
      .from('CustomOrder')
      .update({ status })
      .eq('id', id).select().single();

    if (error) throw error;
    res.json({ message: "Statut mis a jour avec succes.", request: updatedData });

  } catch (error) {
    console.error("Erreur Update Status Custom Order:", error);
    res.status(500).json({ message: "Erreur lors de la mise a jour." });
  }
});

export default router;
```

B. Interfaces et Fonctionnalités Supplémentaires

Annexe B.1 : Tunnel d'achat et Panier

Cette interface permet au client de réviser ses articles avant de procéder au paiement. Le montant total est calculé dynamiquement en fonction des quantités sélectionnées.



FIGURE B.1 – Interface du Panier (Cart) et récapitulatif de commande

Annexe B.2 : Génération de Facture PDF

Exemple de facture générée automatiquement par le système backend une fois la commande validée, prête à être envoyée au client.

DEER GOODS.
Premium Leather Goods
Casablanca, Morocco
Email: leatherstore27@gmail.com

FACTURE
N° Commande : #B2DE6662
Date : 04/06/2026
Paiement : COD

FACTURÉ À :
Nom : moh27sbi
Email : moh27sbi@gmail.com
Téléphone : 0772114755
Adresse : Sbata, Casablanca

Produit	Prix Unitaire	Quantité	Total
Handcrafted Leather Wallet	250.00 DH	1	250.00 DH
Handcrafted Leather Briefcase	550.00 DH	1	550.00 DH

TOTAL À PAYER : 800.00 DH

Merci pour votre confiance ! - DEER GOODS

FIGURE B.2 – Modèle de Facture PDF générée par le système

Annexe B.3 : Version Mobile (Responsive Design)

L'interface de la plateforme s'adapte automatiquement aux différentes tailles d'écran, garantissant une expérience utilisateur optimale sur smartphone.

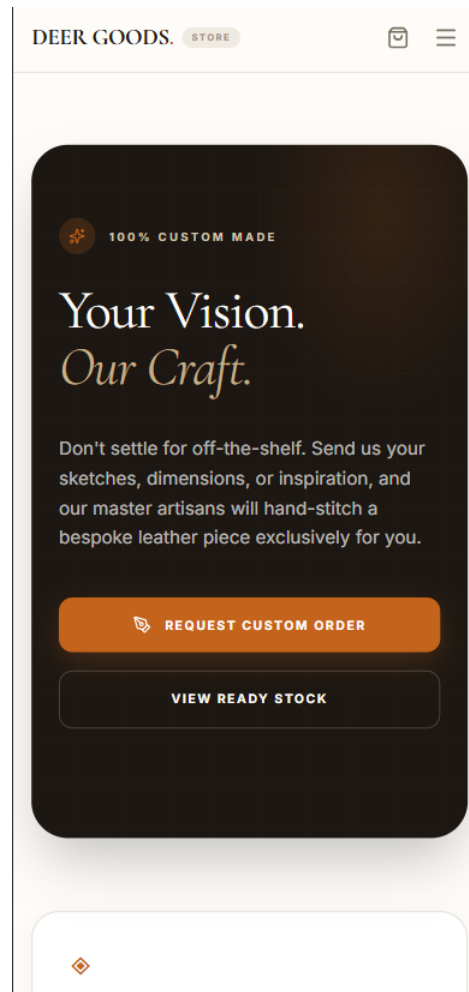


FIGURE B.3 – Aperçu de la page d’accueil sur un terminal mobile (Iphone XR)

Annexe B.4 : Création de Compte et Inscription (Register)

Cette interface permet aux nouveaux utilisateurs de créer un compte sécurisé sur la plateforme en renseignant leurs informations personnelles (nom, email, téléphone et mot de passe). Côté backend, le système valide les données et procède au hachage du mot de passe avant son stockage dans la base de données Supabase.

DEER GOODS.

Begin your journey.
Join the craft.

Create an account to track orders, save your favourite pieces, and receive early access to new collections.

- 100% handmade genuine leather in Morocco
- Custom engraving and personalization
- Lifetime warranty on all stitching
- VIP access to limited edition drops

EST. 2018 — HANDMADE IN MOROCCO

1 ACCOUNT 2 PERSONAL 3 DONE

STEP 1 OF 2

Create account

Join our community of leather enthusiasts.

EMAIL ADDRESS

your@email.com

PASSWORD

Min. 8 characters

CONTINUE →

Already have an account? [Sign In](#)

FIGURE B.4 – Interface de création de compte (Sign Up / Register)